

Laporan Analisis Performa Aplikasi GOD-PBJ

Pertanyaan yang dijawab: Bagaimana agar seluruh halaman aplikasi dapat digunakan secara optimal oleh lebih dari 50 orang secara bersamaan?

Tanggal analisis: 6 Agustus 2026

Fokus utama: Halaman Ringkasan (/ringkasan), dengan pemetaan seluruh halaman lain

Status: Analisis saja — belum ada kode yang diubah

Daftar Isi

1. [Ringkasan Eksekutif](#)
2. [Cara Membaca Laporan Ini](#)
3. [Apa yang Terjadi Saat Halaman Ringkasan Dibuka](#)
4. [Mengapa Sistem Kacau di 50 Pengguna](#)
5. [Empat Akar Masalah di Dalam Database](#)
6. [Masalah Tambahan di Luar Kecepatan](#)
7. [Peta Kondisi Seluruh Halaman](#)
8. [Konsep Solusi](#)
9. [Rencana Kerja Bertahap](#)
10. [Apa yang Terjadi Jika Tidak Dikerjakan](#)
11. [Lampiran A — Metode Pengukuran](#)
12. [Lampiran B — Daftar Berkas Terkait](#)
13. [Lampiran C — Glosarium](#)

1. Ringkasan Eksekutif

Temuan utama

Halaman Ringkasan saat ini **tidak akan sanggup melayani 50 pengguna bersamaan**. Bukan karena datanya terlalu besar — 7.734 paket adalah jumlah yang sangat wajar — melainkan karena **cara**

kerjanya salah secara mendasar:

Setiap kali satu orang membuka halaman, aplikasi mengambil **seluruh isi database** ke browser orang tersebut, lalu menghitung semua angka dari nol di komputer orang itu. Setelah selesai, semuanya dibuang. Orang berikutnya mengulang seluruh proses dari awal.

Ini berarti **biaya sistem berbanding lurus dengan jumlah pengguna**, bukan dengan jumlah data. Pengguna ke-50 memicu kerja yang persis sama beratnya dengan pengguna ke-1, padahal mereka semua melihat angka yang identik.

Angka hasil pengukuran langsung

Saya mengukur langsung ke sistem yang sedang berjalan, bukan memperkirakan:

Yang diukur	Hasil
Jumlah data yang dipindahkan untuk satu kali buka halaman Ringkasan	± 19 MB
Jumlah permintaan ke database untuk satu kali buka halaman	± 26 kali
Waktu tunggu sebelum grafik pertama muncul	± 10 detik
Jumlah baris data yang dikirim ke browser	± 21.700 baris
Jumlah angka yang sebenarnya ditampilkan di layar	± 40 angka + 10 grafik

Dengan kata lain: aplikasi memindahkan data setara **10.000 halaman dokumen Word** untuk menampilkan informasi yang muat dalam **satu lembar kertas**.

Kesimpulan tiga kalimat

1. **Akar masalahnya ada di struktur database**, bukan di tampilan — sehingga memperbaiki tampilan saja tidak akan menolong.
2. **Solusinya sudah diketahui oleh tim sendiri** dan tertulis di catatan kode, hanya belum dikerjakan.
3. **Satu langkah perbaikan saja** (membuat "rekap tersimpan") sudah menyelesaikan sebagian besar masalah, dan bisa dikerjakan dalam waktu sekitar satu hari.

2. Cara Membaca Laporan Ini

Sepanjang laporan ini saya memakai satu perumpamaan yang konsisten, supaya mudah diikuti tanpa perlu latar belakang teknis:

Istilah teknis	Perumpamaan dalam laporan ini
Database	Gudang arsip berisi seluruh berkas paket pengadaan
Query / permintaan data	Petugas yang disuruh mengambil berkas dari gudang
Halaman web	Meja pembaca tempat berkas dihamparkan dan dihitung
Tabel	Rak di dalam gudang
Indeks	Buku daftar isi gudang ("berkas 4521 ada di rak B baris 3")
View	Petunjuk kerja — "ambil dari rak A, gabungkan dengan rak C, lalu jumlahkan"
Materialized view	Rekap jadi yang disimpan di lemari depan, tidak dihitung ulang
Cache	Fotokopi rekap yang dibagikan ke banyak orang sekaligus
Bundle JavaScript	Peralatan yang harus dibawa staf sebelum mulai bekerja

Istilah teknis aslinya tetap saya cantumkan di [Lampiran C — Glosarium](#), untuk keperluan diskusi dengan tim pengembang.

3. Apa yang Terjadi Saat Halaman Ringkasan Dibuka

3.1. Alurnya secara utuh

Ketika seorang pengguna mengetik alamat halaman Ringkasan, inilah yang terjadi berurutan:

Langkah 1 — Browser mengunduh "peralatan kerja"

Sebelum satu pun angka bisa ditampilkan, browser harus mengunduh program aplikasinya dulu. Ukurannya sekitar **1,4 MB**. Di dalamnya termasuk:

- Program pembuat file Excel ($\pm$ 950 KB)
- Program pembuat file PDF ($\pm$ 400 KB)
- Program pengambil gambar layar untuk fitur Cetak ($\pm$ 50 KB)

Masalahnya: **ketiga program itu hanya dipakai kalau pengguna menekan tombol Export atau Cetak**. Sebagian besar pengguna hanya ingin melihat angka, tidak pernah menekan tombol itu — tapi mereka tetap harus mengunduh semuanya lebih dulu. Ibaratnya, staf disuruh membawa mesin fotokopi dan printer ke meja pembaca padahal yang diminta cuma dibacakan angkanya.

Langkah 2 — Halaman tampil kosong dengan tanda "Memuat..."

Saat ini, halaman Ringkasan tidak menyiapkan apa pun di sisi server. Seluruh pekerjaan dimulai **setelah** browser selesai mengunduh peralatan. Jadi pengguna melihat layar kosong berputar, bukan angka.

Langkah 3 — Tiga rombongan pengambilan data berjalan bersamaan

Di sinilah beban sesungguhnya. Ada tiga rombongan petugas yang dikirim ke gudang:

Rombongan A — Data paket gabungan

Rincian	Nilai
Yang diambil	Seluruh paket pengadaan (semua metode, semua satker, semua tahun)
Jumlah baris	7.734
Jumlah bolak-balik ke gudang	8 kali
Ukuran data mentah	3,7 MB
Waktu per kali angkut (hasil ukur)	$\pm$ 1,3 detik

Kenapa 8 kali bolak-balik? Karena gudang membatasi maksimal 1.000 berkas sekali angkut. Jadi petugas harus kembali lagi, lagi, dan lagi.

Yang penting dipahami: petugas ini bekerja **berurutan, bukan bersamaan**. Dia baru berangkat mengambil berkas ke-1001 setelah berkas ke-1000 sampai di meja. Jadi $8 \times 1,3 \text{ detik} = \pm 10 \text{ detik}$ hanya untuk rombongan ini saja.

Rombongan B — Data risiko pengadaan

Rincian	Nilai
Yang diambil	Seluruh data penilaian risiko, termasuk rincian komponennya
Jumlah baris	7.741
Jumlah bolak-balik ke gudang	8 kali
Ukuran data mentah	15 MB ← paling berat
Yang sebenarnya ditampilkan	Dua grafik kecil

Ini rombongan paling boros. Setiap baris data risiko membawa serta **rincian lengkap seluruh komponen penilaiannya** dalam bentuk lampiran panjang. Padahal yang dibutuhkan grafik hanyalah **hitungan berapa banyak paket per tingkat skor** — sekitar 30 angka.

Perumpamaannya: untuk membuat grafik "berapa paket berisiko tinggi, sedang, rendah", petugas mengangkut **seluruh berkas penilaian lengkap beserta seluruh lampirannya** — padahal cukup membaca stempel di sampul depan.

Rombongan C — Data ITKP (Indeks Tata Kelola Pengadaan)

Rincian	Nilai
Yang diambil	Data afirmasi, master data, realisasi PL, realisasi PnL
Jumlah baris	± 6.238
Jumlah bolak-balik ke gudang	± 10 kali
Ukuran data	± 1 MB

Rombongan ini sudah lebih hemat dari yang lain — pengembang sebelumnya sudah memperbaikinya agar sebagian data dipinjam dari Rombongan A, tidak diambil ulang. Ini bukti bahwa tim sudah sadar akan masalah ini dan pernah menanganinya sebagian.

Langkah 4 — Semua dihitung di browser pengguna

Setelah ± 21.700 baris sampai di meja, browser menghitung semuanya sendiri: menjumlahkan pagu, menjumlahkan realisasi, mengelompokkan per metode, per jenis, per satker, menghitung persentase, menyusun peringkat.

Langkah 5 — Setiap kali filter diubah, hitungan diulang dari nol

Ketika pengguna memilih satker tertentu di kotak filter, browser **menghitung ulang seluruh 7.734 baris dari awal**. Ini terjadi setiap kali filter disentuh.

Langkah 6 — Semua dibuang saat pindah halaman

Pengguna pindah ke halaman Tender, lalu kembali ke Ringkasan. Seluruh proses Langkah 3–5 **diulang dari nol**. Tidak ada yang disimpan.

3.2. Rekapitulasi satu kali buka halaman

Rombongan	Baris	Bolak-balik	Data mentah
A — Paket gabungan	7.734	8	3,7 MB
B — Risiko pengadaan	7.741	8	15,0 MB
C — Data ITKP	6.238	± 10	1,0 MB
Total	± 21.700	± 26	± 19,7 MB

Ditambah 1,4 MB peralatan di Langkah 1.

4. Mengapa Sistem Kacau di 50 Pengguna

4.1. Matematikanya sederhana

Jumlah pengguna bersamaan	Permintaan ke gudang	Data diproses gudang
1 orang	26	19 MB
10 orang	260	190 MB
50 orang	± 1.300	± 950 MB

4.2. Tapi gudang hanya punya sekitar 10 pintu

Ini bagian yang menentukan. Database Supabase membatasi berapa banyak permintaan yang bisa dilayani bersamaan — standarnya sekitar **10 sampai 15 "pintu"**.

Ketika 1.300 permintaan datang ke 10 pintu, yang terjadi bukan "semuanya jadi 130 kali lebih lambat". Yang terjadi adalah:

1. Antrean menumpuk
2. Permintaan yang menunggu terlalu lama **dibatalkan otomatis** oleh sistem (istilahnya *timeout*)
3. Pengguna melihat pesan **"Gagal memuat data"**
4. Pengguna menekan tombol **Refresh** — yang mengirim **26 permintaan baru** ke antrean yang sudah penuh
5. Keadaan memburuk, bukan membaik

Poin nomor 4 penting: **reaksi alami pengguna justru memperparah keadaan**. Ini disebut efek bola salju. Sistem tidak melambat secara perlahan lalu pulih — sistem melewati titik tertentu lalu jatuh, dan sulit bangkit selama orang masih menekan Refresh.

4.3. Ini sudah pernah terjadi

Bukan prediksi. Di catatan berkas `sql/add_index_realisasi_dashboard.sql` tertulis:

"fetch E-Purchasing gagal dengan 'canceling statement due to statement timeout' karena view_dashboard_epurchasing_v6 melakukan FULL OUTER JOIN ... tanpa index sama sekali — sehingga tiap panggilan full-scan tabel dasar, dan panggilan itu terjadi berulang karena semua modul Realisasi menarik data lewat loop pagination di sisi client."

Diterjemahkan ke bahasa sehari-hari: **sistem sudah pernah mati karena persis masalah ini**, dan waktu itu penggunaanya masih sedikit. Yang dilakukan saat itu adalah menambal dengan membuat daftar isi (indeks). Tambalan itu menolong, tapi tidak menghilangkan akar masalahnya — sebagaimana akan dijelaskan di bagian berikutnya.

4.4. Kenapa 50 pengguna itu pemborosan yang tidak perlu

Ini yang paling disayangkan dari seluruh temuan:

50 orang yang membuka halaman Ringkasan pada jam yang sama **melihat angka yang persis sama**. Total pagu Kementerian tidak berbeda antara orang ke-1 dan orang ke-50.

Namun sistem menghitungnya **50 kali secara terpisah**, dari nol, dengan hasil yang identik. Ini seperti 50 orang datang ke perpustakaan menanyakan pertanyaan yang sama, lalu pustakawan mengulang riset yang sama 50 kali alih-alih menulis jawabannya sekali di papan pengumuman.

5. Empat Akar Masalah di Dalam Database

Bagian ini menjelaskan **mengapa satu kali pengambilan data saja sudah mahal** — sebelum bicara soal 50 pengguna. Temuan ini berasal dari pembacaan struktur database (`database.types.ts`) dan berkas-berkas SQL di folder `sql/` .

Ini bagian terpenting dari laporan, karena **mengubah urutan prioritas perbaikan**.

5.1. Gudang tidak dirancang, melainkan ditumpuk

Apa yang saya temukan:

Di dalam struktur database, setiap tabel tercatat dengan keterangan `Relationships: []` — artinya **tidak ada satu pun hubungan resmi yang didaftarkan antar tabel**. Nol, di seluruh database.

Apa artinya dalam bahasa sederhana:

Database yang dirancang dengan baik mendaftarkan hubungan antar rak, misalnya: "kolom nomor paket di rak Realisasi **harus** merujuk ke nomor paket yang ada di rak RUP". Pendaftaran ini memberi dua manfaat:

1. **Menjaga kebenaran data** — sistem menolak data realisasi untuk paket yang tidak ada
2. **Mempercepat pencarian** — sistem tahu jalur mana yang sering dilewati, dan menyiapkan jalan pintas

Karena tidak ada satu pun yang didaftarkan di sini, database ini pada dasarnya adalah **kumpulan berkas hasil impor mentah** dari SIRUP dan SPSE — bukan sistem yang dirancang. Daftar isi (indeks) baru dibuat belakangan, **hanya di tempat-tempat yang pernah membuat sistem mati**.

Bukti bahwa ini bukan tuduhan: catatan di `sql/add_index_realisasi_dashboard.sql` menulis sendiri bahwa *"sebelumnya tidak ada satupun index di tabel dasar tersebut"*.

Dampak praktisnya: untuk banyak pencarian, petugas gudang harus **membuka seluruh rak satu per satu**, karena tidak ada daftar isi yang bisa dipakai.

5.2. Nomor paket ditulis dengan format berbeda-beda

Ini akar masalah yang paling luas dampaknya.

Apa yang saya temukan:

Nomor paket (`kd_rup`) adalah kunci yang menyambungkan seluruh sistem — RUP, realisasi, kurasi AI, penilaian risiko, semuanya dihubungkan lewat nomor ini. Tapi di dalam database, formatnya **tidak seragam**:

Rak (tabel)	Nama kolom	Format
Paket penyedia terumumkan	<code>kd_rup</code>	Angka
Kurasi AI	<code>kd_rup</code>	Angka
Anggaran penyedia	<code>kd_rup</code>	Angka
E-Purchasing	<code>rup_code</code>	Teks
Non-tender selesai	<code>kd_rup</code>	Teks
Pencatatan realisasi non-tender	<code>kd_rup_paket</code>	Teks
Tender selesai	<code>kd_rup_paket</code>	Teks
Risiko pengadaan	<code>kd_rup</code>	Teks

Kenapa ini masalah:

Bagi komputer, angka `4521` dan teks `"4521"` adalah **dua benda yang sama sekali berbeda** — seperti angka 5 dan huruf "lima". Tidak bisa langsung dibandingkan.

Jadi setiap kali sistem menyambungkan dua rak, ia harus **menerjemahkan formatnya dulu, baris demi baris, sebanyak ribuan kali**.

Dan inilah bagian yang paling merugikan: begitu data diterjemahkan, **daftar isi gudang menjadi tidak bisa dipakai** — karena daftar isi disusun berdasarkan format aslinya. Petugas kembali harus membuka seluruh rak.

Cara tim mengakalinya, dan mengapa itu rapuh:

Tim membuat daftar isi khusus yang **sudah dalam format terjemahan**. Ini berhasil — tapi hanya bekerja jika cara pencariannya ditulis **persis sama, karakter demi karakter**.

Catatan tim sendiri di `sql/add_index_realisasi_dashboard.sql` menyatakan:

"Postgres hanya memakai index kalau ekspresinya cocok persis dengan yang ada di query."

Dan ini **sudah pernah gagal**. Berkas `sql/add_index_ltrim_satker.sql` mendokumentasikan kejadiannya:

"index exact-match lama TIDAK lagi dipakai Postgres, karena ekspresi JOIN kini dibungkus LTRIM. Tanpa index yang cocok, view kembali full-scan dan bisa memicu 'statement timeout' seperti sebelumnya."

Diterjemahkan: **seseorang mengubah cara pencarian sedikit saja, daftar isi langsung tidak terpakai, dan sistem mati.**

Artinya kondisi sekarang adalah **tambalan yang sangat rapuh**. Setiap kali ada pengembang yang menyentuh bagian ini di kemudian hari, ada risiko nyata sistem mati lagi tanpa peringatan.

Ada satu kasus yang bahkan tidak bisa ditambal sama sekali:

Di berkas `sql/create_view_dashboard_pengadaan_langsung.sql` baris 80, penyambungan datanya berbunyi (disederhanakan):

"Ambil bagian nomor paket **sebelum tanda titik-koma**, lalu cocokkan."

Ini karena ada baris yang nomor paketnya berisi **dua nomor sekaligus**, ditulis sebagai `"4521;4522"`. Kolom `is_multiple_rup` di database mengonfirmasi hal ini memang terjadi.

Pemotongan seperti ini **tidak bisa dibantu daftar isi apa pun** yang saat ini ada. Jadi cabang Pengadaan Langsung — yang berisi **6.023 baris**, terbesar kedua — **dijamin selalu dibuka seluruh raknya** setiap kali halaman Ringkasan dimuat.

Inilah penjelasan atas hasil pengukuran saya:

Yang diminta	Waktu
Berkas ke-1 sampai ke-1000	1,290 detik
Berkas ke-3001 sampai ke-4000	1,316 detik
Berkas ke-7001 sampai ke-7734	1,477 detik

Ketiganya **hampir sama**. Kalau gudang punya daftar isi yang berfungsi, mengambil berkas terakhir seharusnya jauh lebih cepat daripada memulai dari awal. Kenyataannya sama saja — karena **setiap permintaan membuka seluruh gudang dari nol**.

5.3. Nilai rupiah disimpan sebagai tulisan, bukan sebagai angka

Ini temuan yang paling berisiko, karena bisa mematikan halaman secara mendadak.

Apa yang saya temukan:

Di beberapa rak penting, nilai uang **tidak disimpan sebagai angka**, melainkan sebagai teks — seperti mencatat "1.500.000" di buku tulis, alih-alih memasukkannya ke kalkulator.

Rak (tabel)	Kolom yang seharusnya angka, tapi disimpan sebagai teks
Non-tender selesai	pagu, HPS, nilai kontrak, nilai negosiasi, nilai penawaran, nilai terkoreksi, nilai PDN kontrak, nilai UMK kontrak, tahun anggaran
Pencatatan realisasi non-tender	nilai realisasi , pagu, tahun anggaran
Master data PN	pagu, realisasi anggaran, target volume, realisasi volume, selisih pagu, persentase capaian
Master data RO	nilai paket
Anggaran penyedia & swakelola	tahun anggaran
Tender selesai	tahun anggaran

Bagaimana sistem menyiasatinya:

Di berkas `sql/create_view_dashboard_pengadaan_langsung.sql` baris 8, perintah penjumlahannya berbunyi (disederhanakan):

```
"Untuk setiap baris: baca tulisannya, ganti semua tanda koma menjadi titik, ubah jadi angka, baru jumlahkan."
```

Kenapa ini mahal:

Pembersihan dan penerjemahan itu dilakukan **satu per satu untuk setiap baris**, setiap kali ada orang membuka halaman. Untuk 6.023 baris Pengadaan Langsung, itu 6.023 kali pembersihan teks — **dikali 8 kali bolak-balik, dikali jumlah pengguna**.

Ibaratnya: setiap kali ditanya "berapa total realisasi?", petugas harus mengambil 6.023 lembar catatan tulisan tangan, **membaca dan mengetik ulang setiap angkanya ke kalkulator**, baru menjumlahkan. Setiap kali. Untuk setiap orang.

Dan inilah risiko terbesarnya:

Kalau ada **satu saja** baris yang tulisannya tidak rapi — spasi nyasar, tanda strip - untuk nilai kosong, tulisan "N/A", atau format 1,500,000 bergaya Amerika — maka penerjemahan itu **gagal**.

Yang terjadi bukan angka yang salah. Yang terjadi adalah **halaman Ringkasan mati total untuk semua orang**, dengan pesan error, sampai baris bermasalah itu ditemukan dan diperbaiki secara manual di database.

Karena data ini masuk dari impor SIRUP/SPSE secara berkala dan **tidak ada pemeriksaan format** (ingat: tidak ada satu pun aturan hubungan yang didaftarkan — poin 5.1), **satu file impor yang formatnya sedikit berbeda bisa mematikan dashboard seluruh Kementerian**.

Ini bukan risiko teoretis. Ini menunggu waktu.

5.4. Berkas semua tahun ditumpuk jadi satu

Apa yang saya temukan:

Setiap rak di database punya kolom tahun_anggaran . Tapi **tidak ada satu pun** pengambilan data di aplikasi yang menyaring berdasarkan tahun. Semuanya diambil, lintas tahun.

Kenapa ini penting:

Ini adalah masalah yang **berjalan sendiri, terpisah dari jumlah pengguna**. Halaman Ringkasan akan melambat setiap tahun anggaran baru, **bahkan jika penggunanya tetap satu orang**.

Perkiraan pertumbuhan, dengan asumsi volume pengadaan per tahun setara:

Tahun	Perkiraan jumlah baris	Perkiraan data dipindahkan
2026 (sekarang)	7.734	19 MB
2027	± 15.500	± 38 MB
2028	± 23.000	± 57 MB
2029	± 31.000	± 76 MB

Dan ingat, waktu tunggu tidak naik secara lurus — ia naik lebih tajam, karena semakin banyak baris berarti semakin banyak pembersihan teks (poin 5.3) dan semakin panjang pencarian tanpa daftar isi (poin 5.2).

Kabar baiknya: ini masalah termurah untuk diperbaiki dalam seluruh laporan. Cukup menambahkan penyaring tahun anggaran pada setiap pengambilan data. Perkiraan waktu kerja: **2 jam**.

Kabar buruknya: kalau tidak dikerjakan sekarang, di tahun anggaran berikutnya semua perbaikan lain akan terasa setengah sia-sia, karena datanya sudah dua kali lipat.

5.5. Ringkasan keempat akar masalah

#	Akar masalah	Dampak	Bisa ditambah?	Perbaikan permanen
5.1	Tidak ada hubungan antar rak yang didaftarkan	Pencarian lambat, data tidak terjaga	Sebagian (indeks manual)	Daftarkan hubungan resmi
5.2	Format nomor paket tidak seragam	Daftar isi tidak terpakai → buka seluruh rak	Ya, tapi sangat rapuh	Seragamkan formatnya
5.3	Nilai uang disimpan sebagai tulisan	Lambat + risiko halaman mati total	Tidak	Ubah jadi tipe angka
5.4	Data semua tahun ditumpuk	Melambat setiap tahun	Ya, mudah	Saring per tahun

6. Masalah Tambahan di Luar Kecepatan

6.1. Risiko angka salah tanpa peringatan — PENTING

Ini temuan yang menurut saya paling perlu segera ditindak, terlepas dari soal 50 pengguna.

Apa masalahnya:

Ketika petugas mengambil berkas bertahap (1.000 per angkut), **tidak ada instruksi urutan yang pasti** kepada gudang. Perintahnya hanya "berikan 1.000 berkas berikutnya" — tanpa menyebutkan berdasarkan urutan apa.

Dalam kondisi gudang sepi, ini biasanya aman: gudang cenderung memberikan dalam urutan yang sama.

Tapi justru saat gudang ramai — yaitu persis saat 50 orang mengakses bersamaan — gudang bisa mengubah cara kerjanya untuk mengejar kecepatan. Ketika itu terjadi, urutannya bisa berubah di tengah jalan, sehingga:

- **Berkas yang sama diberikan dua kali** → total pagu jadi lebih besar dari seharusnya
- **Sebagian berkas terlewat** → total pagu jadi lebih kecil dari seharusnya

Kenapa ini berbahaya:

Tidak ada pesan error. Tidak ada peringatan. Tidak ada tanda apa pun.

Angkanya hanya salah.

Dan karena halaman ini dipakai untuk pelaporan capaian realisasi Kementerian, angka yang salah bisa terbawa ke dokumen resmi tanpa ada yang menyadarinya.

Di mana masalah ini ada: di **seluruh 9 halaman** yang mengambil data bertahap — Ringkasan, Risiko, Tender, E-Purchasing, Pengadaan Langsung, Penunjukan Langsung, Swakelola, Rencana Pengadaan, dan ITKP.

Perbaikannya sangat sederhana: tambahkan instruksi urutan pada setiap pengambilan data. Perkiraan waktu kerja: **setengah hari untuk seluruh halaman.**

Rekomendasi saya: kerjakan ini lebih dulu daripada yang lain, karena murah, cepat, dan menyangkut kebenaran angka — bukan sekadar kecepatan.

6.2. Beban di komputer pengguna

Data 19 MB yang diterima browser, setelah diolah, memakan **sekitar 40–60 MB memori** di komputer pengguna. Ditambah:

- Setiap kali filter diubah, seluruh 7.734 baris dihitung ulang dari nol
- Aplikasi menyiapkan **dua salinan lengkap** data untuk fitur Export (satu untuk "semua data", satu untuk "data terfilter") — **meskipun jendela Export belum pernah dibuka**

Dampak nyatanya: di laptop kantor dengan spesifikasi standar, tab browser akan terasa berat, filter terasa lambat merespons, dan dalam kondisi tertentu tab bisa berhenti merespons.

Perlu dicatat: ini terjadi **di komputer pengguna masing-masing**, jadi tidak akan terlihat di pemantauan server. Keluhan akan muncul sebagai "aplikasinya berat" tanpa jejak teknis apa pun.

6.3. Peralatan berat yang jarang dipakai

Seperti dijelaskan di bagian 3.1 Langkah 1: setiap pengunjung mengunduh **± 1,4 MB** program, di mana sekitar **1,4 MB di antaranya** adalah program Export Excel, Export PDF, dan Cetak Gambar.

Program-program ini hanya berguna kalau tombolnya ditekan. Untuk pengguna yang hanya ingin melihat angka — yang kemungkinan besar adalah mayoritas — ini murni pemborosan.

Perbaikannya: ubah agar program tersebut baru diunduh **saat tombolnya ditekan**. Halaman akan terbuka dengan jauh lebih ringan (perkiraan: dari 1,4 MB turun ke sekitar 350 KB), dan pengguna yang menekan Export hanya menunggu tambahan satu-dua detik saat menekannya.

6.4. Catatan keamanan — di luar lingkup pertanyaan, tapi perlu diketahui

Saat melakukan pengukuran, saya menemukan hal berikut:

Saya berhasil membaca **seluruh 7.734 data paket pengadaan** dan **seluruh 7.741 data penilaian risiko** — **tanpa login sama sekali** — hanya dengan menggunakan kunci akses publik yang memang tertanam di dalam halaman web dan dapat dilihat oleh siapa pun yang membuka menu pengembang di browser.

Konteks tambahan:

Pembatasan akses berdasarkan peran (misalnya PPK hanya boleh melihat satkernya sendiri) saat ini diterapkan **di lapisan tampilan** — artinya seluruh data tetap dikirim utuh ke browser pengguna, lalu sebagiannya disembunyikan dari layar. Data yang "disembunyikan" itu tetap ada di dalam browser dan dapat dilihat oleh siapa pun yang tahu caranya.

Hal ini terlihat jelas dari catatan di kode aplikasi sendiri, yang menyatakan bahwa pembatasan peran PPK *"ditegakkan pada tingkat tampilan ... bukan dengan memotong dataset"*.

Yang perlu dilakukan: ini harus diperiksa oleh pihak yang menangani keamanan sistem. Saya tidak melakukan perubahan apa pun terkait ini, dan tidak memasukkannya ke rencana kerja di bawah — karena ini keputusan di luar ranah teknis performa.

Catatan penting: perbaiki performa yang saya usulkan (memindahkan perhitungan ke server) **kebetulan juga memperbaiki masalah ini sebagian**, karena data mentah tidak lagi dikirim ke browser — yang dikirim hanya angka hasil rekap. Jadi kedua kepentingan ini sejalan.

7. Peta Kondisi Seluruh Halaman

Seluruh 9 halaman aplikasi memakai pola kerja yang sama persis: ambil semua data ke browser, hitung di browser. Namun tingkat keparahannya berbeda-beda, tergantung banyaknya data.

Halaman	Jumlah baris	Bolak-balik	Kondisi	Prioritas
Ringkasan	± 21.700 (3 sumber)	± 26	● Kritis	1
Risiko Pengadaan	7.741	8	● Kritis	2
Pengadaan Langsung	6.023	7	● Kritis	3
ITKP	± 6.238	± 10	● Sedang	4
E-Purchasing	1.521	2	● Sedang	5
Rencana Pengadaan	bervariasi	bervariasi	● Sedang	6
Penunjukan Langsung	83	1	● Aman	7
Tender	64	1	● Aman	7
Swakelola	43	1	● Aman	7

Cara membaca tabel ini:

- **Kritis** — akan bermasalah nyata di 50 pengguna, perlu perbaikan struktural
- **Sedang** — akan terasa lambat tapi kemungkinan besar tidak mati
- **Aman** — datanya kecil, tidak bermasalah dari sisi kecepatan

Penting: halaman berstatus ● Aman **tetap perlu** perbaikan yang dijelaskan di bagian 6.1 (risiko angka salah). Halaman-halaman itu aman dari sisi kecepatan, tapi tidak kebal dari kemungkinan angka keliru. Untungnya perbaikannya murah dan bisa sekalian dikerjakan.

Catatan tentang Pengadaan Langsung: halaman ini masuk kategori kritis bukan hanya karena 6.023 barisnya, tapi karena ia mengandung **dua akar masalah terberat sekaligus** — pemotongan nomor paket yang tidak bisa dibantu daftar isi (bagian 5.2), dan pembersihan teks nilai uang per baris

(bagian 5.3). Dan karena halaman Ringkasan juga menarik data yang sama, **memperbaiki cabang ini menguntungkan dua halaman sekaligus.**

8. Konsep Solusi

8.1. Prinsipnya satu kalimat

Berhenti menghitung ulang. Mulai menyiapkan rekap.

8.2. Bagaimana bentuknya

Kondisi sekarang:

```
Pengguna buka halaman
↓
Petugas ke gudang (26 kali bolak-balik)
↓
Angkut 21.700 berkas ke meja pembaca
↓
Terjemahkan format nomor, bersihkan tulisan angka
↓
Hitung semuanya dari nol
↓
Tampilkan 40 angka
↓
Buang semuanya
↓
Pengguna berikutnya → ulangi dari atas
```

Kondisi yang diusulkan:

Data baru masuk dari SIRUP (misalnya sekali sehari)



SEKALI SAJA: hitung seluruh rekap, simpan di "lemari depan"



Pengguna buka halaman



Ambil satu lembar rekap dari lemari depan (1 kali, ± 10 KB)



Tampilkan



Pengguna berikutnya → cukup baca fotokopi rekap yang sama

8.3. Dua lapisan perbaikan, dan mengapa keduanya dibutuhkan

Lapisan 1 — Rekap tersimpan *(mengurangi biaya per permintaan)*

Seluruh pekerjaan berat — membuka rak, menerjemahkan format nomor, membersihkan tulisan angka, menggabungkan lima sumber data — dilakukan **sekali saja setiap kali data baru masuk**, lalu hasilnya disimpan sebagai berkas jadi.

Setelah ini, satu permintaan data yang tadinya butuh $\pm 1,3$ detik menjadi sekitar **0,015 detik** — sekitar **90 kali lebih cepat**.

Lapisan 2 — Fotokopi rekap *(mengurangi jumlah permintaan)*

Kalau 50 orang meminta rekap dalam rentang 5 menit yang sama, sistem cukup mengambilnya **satu kali**, lalu membagikan salinannya ke semua orang.

Setelah ini, **50 pengguna membebani database sama ringannya dengan 1 pengguna**.

8.4. Mengapa urutannya penting

Ini poin yang membuat saya merevisi rekomendasi awal:

Jika hanya mengerjakan Lapisan 2 tanpa Lapisan 1, hasilnya **tidak sebaik yang diharapkan**. Sebab yang di-"fotokopi" adalah hasil perhitungan yang tetap mahal — sekitar 1,3 detik. Orang pertama setiap 5 menit tetap menunggu lama, dan jika beberapa permintaan pertama datang bersamaan, antreannya tetap menumpuk.

Kesimpulan: Lapisan 1 harus dikerjakan lebih dulu. Lapisan 2 melipatgandakan manfaatnya, tapi tidak bisa menggantikannya.

8.5. Ini bukan ide baru — tim sudah menyimpulkan hal yang sama

Tiga bukti dari dalam proyek ini sendiri:

Bukti 1. Catatan penutup di `sql/add_index_realisasi_dashboard.sql` :

"kalau timeout masih terjadi setelah index ini terpasang, langkah berikutnya adalah mengubah view_paket_penyedia_master_data dan view_dashboard_epurchasing_v6 menjadi materialized view (refresh berkala)."

"Materialized view dengan refresh berkala" **adalah persis** yang saya sebut "rekap tersimpan yang diperbarui saat data masuk". Rencananya sudah ditulis. Tinggal dikerjakan.

Bukti 2. Di database sudah ada `view_dashboard_keterisian_sirup_eselon1` — sebuah rekap tingkat Eselon I yang sudah dihitung di sisi database. Jadi pola "hitung di database, bukan di browser" **sudah dipakai** di proyek ini, hanya belum diterapkan untuk halaman Ringkasan.

Bukti 3. Di database sudah ada fungsi `get_rup_history` — sebuah perhitungan yang dijalankan di sisi database dan mengembalikan hasil jadi. Jadi kemampuan teknis untuk membuat "rekap di database" **sudah ada dan sudah terbukti dipakai**. Yang saya usulkan bukan kemampuan baru, hanya perluasan dari yang sudah ada.

Kesimpulan: rekomendasi ini bukan perombakan arsitektur yang asing bagi tim. Ini melanjutkan arah yang sudah ditetapkan tim sendiri.

9. Rencana Kerja Bertahap

Delapan langkah, disusun berdasarkan **dampak per jam kerja**. Setiap langkah dijelaskan: apa yang dikerjakan, mengapa perlu, hasil yang diharapkan, dan cara memastikan berhasil.

Langkah 1 — Buat "rekap tersimpan"

Perkiraan waktu	1 hari
Dampak	★★★★★ Terbesar

Risiko pengerjaan	Rendah — tidak mengubah tampilan sama sekali

Apa yang dikerjakan:

Membuat versi "sudah jadi" dari gabungan data paket, yang disimpan permanen di database dan diperbarui setiap kali data baru masuk dari SIRUP.

Mengapa ini nomor satu:

Karena ini satu-satunya langkah yang menyerang **akar** masalah, bukan gejalanya. Setelah ini:

- Penerjemahan format nomor paket (bagian 5.2) → dibayar sekali per pembaruan data, bukan setiap permintaan
- Pembersihan tulisan nilai uang (bagian 5.3) → sama, sekali saja
- Pencarian tanpa daftar isi → tidak relevan lagi, karena rekapnya sudah jadi dan bisa diberi daftar isi yang sederhana dan kokoh

Hasil yang diharapkan:

Ukuran	Sebelum	Sesudah
Waktu per permintaan	± 1,3 detik	± 0,015 detik
Beban database saat 50 pengguna	Antre & mati	Ringan

Cara memastikan berhasil:

Ukur ulang waktu pengambilan data seperti di [Lampiran A](#). Angkanya harus turun dari sekitar 1,3 detik menjadi di bawah 0,1 detik.

Catatan penting:

Perlu ditetapkan **kapan rekap diperbarui**. Karena data ini berasal dari impor berkala SIRUP/SPSE (bukan data yang berubah setiap detik), memperbarui rekap **setiap selesai impor** sudah sangat memadai. Jika impor dilakukan harian, maka rekap diperbarui harian.

Langkah 2 — Saring data berdasarkan tahun anggaran

Perkiraan waktu	2 jam

Dampak	☆☆☆ Besar, dan makin besar seiring waktu
Risiko pengerjaan	Rendah, tapi perlu keputusan kebijakan

Apa yang dikerjakan:

Menambahkan penyaring tahun anggaran pada setiap pengambilan data, sehingga aplikasi hanya menarik data tahun berjalan.

Mengapa perlu:

Seperti dijelaskan di bagian 5.4, tanpa ini halaman akan melambat **setiap tahun anggaran baru**, tanpa ada penambahan pengguna sama sekali.

Yang perlu diputuskan lebih dulu (bukan keputusan teknis):

1. Apakah halaman Ringkasan hanya menampilkan **tahun berjalan**, atau perlu ada pilihan tahun?
2. Jika perlu pilihan tahun, apakah perlu ada tampilan **perbandingan antar tahun**?
3. Data tahun lama tetap disimpan (tidak dihapus) — hanya tidak ikut ditarik kecuali diminta

Saya menyarankan: **tampilkan tahun berjalan sebagai bawaan, sediakan kotak pilihan tahun di sebelah filter Satker**. Ini paling sederhana dan sesuai dengan cara laporan pengadaan biasanya dibaca.

Hasil yang diharapkan:

Belum terasa di 2026 (karena datanya memang baru satu tahun), tapi **mencegah** pelambatan 2× di 2027 dan 3× di 2028.

Langkah 3 — Halaman mengambil angka jadi, bukan menghitung sendiri

Perkiraan waktu	1 hari
Dampak	☆☆☆☆ Sangat besar
Risiko pengerjaan	Sedang — perlu pengujian agar angkanya sama persis
Prasyarat	Langkah 1 harus selesai lebih dulu

Apa yang dikerjakan:

Membuat satu perintah di database yang langsung mengembalikan **seluruh angka jadi** yang dibutuhkan halaman Ringkasan — total pagu, total realisasi, rincian per metode, per jenis, per satker, dan seterusnya.

Halaman kemudian cukup memanggil perintah itu sekali, dan langsung menampilkan hasilnya.

Mengapa harus setelah Langkah 1:

Kalau perintah ini dibuat di atas data yang **belum** direkap, maka setiap pemanggilannya tetap membayar biaya penuh ($\pm 1,3$ detik) — hanya sekali alih-alih 8 kali. Itu perbaikan 8 kali lipat, yang bagus tapi jauh dari cukup.

Kalau dibuat **di atas rekap** dari Langkah 1, hasilnya perbaikan sekitar **700 kali lipat**.

Hasil yang diharapkan:

Ukuran	Sebelum	Sesudah
Bolak-balik ke gudang	26 kali	1 kali
Data dipindahkan	19 MB	± 10 KB
Baris dikirim ke browser	21.700	± 50
Waktu tunggu	± 10 detik	di bawah 1 detik

Cara memastikan berhasil:

Bandingkan setiap angka di halaman Ringkasan sebelum dan sesudah perubahan — Total Pagu, Total Realisasi, jumlah paket per metode, dan seterusnya. **Semua harus sama persis**. Kalau ada yang berbeda, itu petunjuk adanya perbedaan cara hitung yang perlu ditelusuri (dan bisa jadi justru menemukan kekeliruan yang selama ini tidak disadari).

Catatan: fitur Export dan tabel rincian (daftar paket anomali, daftar paket "Tidak Akurat") tetap perlu data baris per baris. Keduanya diambil **terpisah dan hanya saat dibutuhkan** — misalnya saat tombol Export ditekan, atau saat filter satker diaktifkan. Bukan ikut terangkut di awal seperti sekarang.

Langkah 4 — "Fotokopi" rekap untuk banyak pengguna

Perkiraan waktu	½ hari
Dampak	★★★★★ Inilah yang menjawab pertanyaan "50 pengguna"
Risiko pengerjaan	Sedang — perlu membaca dokumentasi versi Next.js yang dipakai
Prasyarat	Langkah 3 sebaiknya selesai

Apa yang dikerjakan:

Menyimpan hasil rekap di sisi server aplikasi untuk jangka waktu tertentu (misalnya 5–15 menit), sehingga permintaan berikutnya dalam rentang itu dilayani dari simpanan, tanpa menyentuh database.

Selain itu, halaman disiapkan agar **angka sudah terisi sejak halaman pertama kali tampil**, bukan kosong lalu diisi belakangan.

Mengapa ini yang benar-benar menjawab pertanyaan awal:

Setelah langkah ini, hubungan antara jumlah pengguna dan beban database **terputus**:

Jumlah pengguna	Permintaan ke database (per 5 menit)
1 orang	1
50 orang	1
200 orang	1

Ini yang mengubah pertanyaan "bagaimana agar kuat 50 orang" menjadi tidak relevan lagi — karena 50 orang tidak lebih berat daripada 1 orang.

Manfaat tambahan:

Halaman tampil dengan angka sudah terisi sejak awal, bukan layar kosong berputar. Perbedaan yang langsung terasa oleh pengguna.

Catatan pelaksanaan:

Berkas `AGENTS.md` di proyek ini menyatakan bahwa versi Next.js yang dipakai memiliki perubahan besar dibanding versi umum. **Dokumentasi versi tersebut harus dibaca lebih dulu** sebelum

menulis kode untuk langkah ini, karena cara kerja penyimpanan sementara kemungkinan berbeda dari yang biasa dipakai.

Langkah 5 — Rekap terpisah untuk data risiko

Perkiraan waktu	½ hari
Dampak	★★★★ Besar
Risiko pengerjaan	Rendah

Apa yang dikerjakan:

Membuat rekap terpisah berisi hitungan skor risiko per komponen, sehingga dua grafik risiko di halaman Ringkasan tidak lagi menarik seluruh rincian penilaian.

Mengapa perlu terpisah dari Langkah 1:

Karena sumbernya beda rak, dan karena inilah **penyumbang beban terbesar tunggal** di halaman Ringkasan: 15 MB dari total 19 MB, atau sekitar **79% dari seluruh beban halaman**, hanya untuk dua grafik.

Penjelasan mengapa data ini luar biasa berat:

Setiap baris penilaian risiko menyimpan **empat lampiran panjang**: rincian komponen penilaian, riwayat revisi, rujukan transaksi, dan penanda kualitas data. Lampiran-lampiran ini disimpan terpisah dari baris utamanya di dalam database, sehingga mengambilnya berarti **dua kali kerja per baris** — ambil barisnya, lalu ambil lampirannya, lalu buka lampirannya.

Dikali 7.741 baris. Setiap kali ada orang buka halaman Ringkasan.

Padahal yang dibutuhkan grafik hanyalah hitungan: berapa paket dengan skor 3, berapa skor 2, berapa skor 1, per jenis komponen. **Sekitar 30 angka.**

Hasil yang diharapkan:

Ukuran	Sebelum	Sesudah
Data dipindahkan	15 MB	± 3 KB
Baris dikirim	7.741	± 30

Ukuran	Sebelum	Sesudah
Bolak-balik	8 kali	1 kali

Langkah 6 — Perbaiki cara pengambilan data bertahap

Perkiraan waktu	½ hari (seluruh halaman)
Dampak	★ ★ ★ Menyangkut kebenaran angka , bukan kecepatan
Risiko pengerjaan	Sangat rendah

Apa yang dikerjakan:

Tiga hal kecil di seluruh 9 halaman:

1. **Menambahkan instruksi urutan** pada setiap pengambilan data bertahap → menutup risiko angka salah yang dijelaskan di bagian 6.1
2. **Menyebutkan kolom yang dibutuhkan** alih-alih meminta "semua kolom" → beberapa halaman saat ini menarik puluhan kolom yang tidak pernah ditampilkan
3. **Memperbaiki cara menggabungkan hasil** → cara yang dipakai sekarang menyalin ulang seluruh data setiap kali ada tambahan, yang menjadi sangat boros pada data besar

Mengapa saya sarankan dikerjakan lebih awal meski dampak kecepatannya kecil:

Karena ini menyangkut **kebenaran angka**, bukan kenyamanan. Angka realisasi yang keliru bisa terbawa ke laporan resmi. Dan perbaikannya sangat murah — setengah hari untuk seluruh aplikasi.

Saran praktis: kerjakan langkah ini **bersamaan dengan Langkah 2**, karena keduanya menyentuh berkas yang sama dan sama-sama singkat. Sekali kerja, dua manfaat.

Langkah 7 — Tunda pemuatan fitur Export dan Cetak

Perkiraan waktu	½ hari
Dampak	★ ★ ★ Terasa langsung oleh pengguna

Risiko pengerjaan	Rendah

Apa yang dikerjakan:

Mengubah agar program Export Excel, Export PDF, dan Cetak Gambar **baru diunduh saat tombolnya ditekan**, bukan saat halaman dibuka.

Hasil yang diharapkan:

Ukuran	Sebelum	Sesudah
Program diunduh saat buka halaman	± 1,4 MB	± 350 KB

Efek sampingnya: pengguna yang menekan tombol Export akan menunggu tambahan sekitar 1–2 detik saat menekannya. Ini pertukaran yang menguntungkan: **semua** pengguna diuntungkan setiap kali membuka halaman, sementara **sebagian kecil** pengguna menunggu sedikit lebih lama sekali saat mengeksplor.

Catatan: langkah ini sepenuhnya berdiri sendiri. Bisa dikerjakan kapan saja, oleh siapa saja, tanpa menunggu langkah lain — cocok dijadikan pekerjaan sisipan.

Langkah 8 — Rapikan format nomor paket dan tipe data nilai uang

Perkiraan waktu	3–5 hari
Dampak	★★★★★ Satu-satunya perbaikan permanen
Risiko pengerjaan	Tinggi — menyentuh struktur data inti

Apa yang dikerjakan:

Dua hal mendasar:

1. **Menyeragamkan format nomor paket** di seluruh rak — satu format, satu tipe. Termasuk memisahkan kasus "4521;4522" menjadi baris tersendiri agar tidak perlu dipotong-potong saat pencarian.
2. **Mengubah kolom nilai uang dari tulisan menjadi angka** — dengan pembersihan data satu kali di awal, dan pemeriksaan format saat impor berikutnya agar tidak terulang.

Mengapa ini disebut satu-satunya perbaikan permanen:

Langkah 1–7 semuanya bersifat **menambal**. Efektif, tapi tidak menghilangkan sumbernya. Selama:

- Nomor paket masih berbeda format antar rak, dan
- Nilai uang masih berupa tulisan

maka:

- Setiap pengembang yang menyentuh bagian ini di masa depan **berisiko mematikan sistem tanpa sadar** — persis seperti yang sudah terdokumentasi di `sql/add_index_ltrim_satker.sql`
- **Satu file impor dengan format sedikit berbeda bisa mematikan dashboard Kementerian** (bagian 5.3)

Langkah 8 menghapus **seluruh kelas masalah ini** sekaligus.

Mengapa ditaruh terakhir meski dampaknya besar:

Karena risikonya paling tinggi dan waktunya paling lama. Langkah 1–7 memberi kelegaan cepat dengan risiko rendah. Langkah 8 sebaiknya dikerjakan **setelah sistem stabil**, dengan waktu yang memadai, dan dengan pengujian menyeluruh.

Saran pelaksanaan: kerjakan bertahap, satu rak per kali, dengan cadangan data dan kemampuan mengembalikan ke kondisi semula. Jangan sekaligus.

9.1. Ringkasan rencana kerja

Langkah	Pekerjaan	Waktu	Dampak	Risiko
1	Buat rekap tersimpan	1 hari	★★★★★	Rendah
2	Saring per tahun anggaran	2 jam	★★★	Rendah
3	Halaman ambil angka jadi	1 hari	★★★★	Sedang
4	Fotokopi rekap (cache)	½ hari	★★★★★	Sedang
5	Rekap data risiko	½ hari	★★★★	Rendah
6	Perbaiki pengambilan bertahap	½ hari	★★★	Sangat rendah
7	Tunda muat Export & Cetak	½ hari	★★★	Rendah

Langkah	Pekerjaan	Waktu	Dampak	Risiko
8	Rapikan format data inti	3–5 hari	★★★★★	Tinggi
	Total	7–9 hari		

9.2. Jika waktu terbatas

Jika hanya sanggup	Kerjakan	Hasilnya
1 langkah	Langkah 1	Beban database turun drastis; 50 pengguna kemungkinan besar tertangani
2 langkah	Langkah 1 + 4	Pertanyaan "50 pengguna" selesai sepenuhnya
Setengah hari saja	Langkah 6	Menutup risiko angka salah di laporan resmi
3 hari	Langkah 1, 2, 3, 4, 6	Aplikasi ringan, angka terjamin, aman untuk tahun-tahun berikutnya

9.3. Urutan pengerjaan yang saya sarankan

Minggu 1

- └ Hari 1 : Langkah 6 (perbaiki pengambilan bertahap) + Langkah 2 (saring tahun)
→ murah, cepat, langsung menutup risiko angka salah
- └ Hari 2 : Langkah 1 (rekap tersimpan)
→ perbaiki terbesar, tidak mengubah tampilan sama sekali
- └ Hari 3 : Langkah 3 (halaman ambil angka jadi)
- └ Hari 4 pagi : Langkah 4 (fotokopi rekap)
→ di titik ini, pertanyaan "50 pengguna" sudah terjawab
- └ Hari 4 sore : Langkah 5 (rekap data risiko)
- └ Hari 5 : Langkah 7 (tunda muat Export) + pengujian menyeluruh

Kemudian, terjadwal terpisah

- └ Langkah 8 : Rapikan format data inti (3–5 hari, risiko tinggi)

Alasan Langkah 6 ditaruh paling depan meski dampak kecepatannya kecil: karena ia menyangkut kebenaran angka, bukan kenyamanan — dan biayanya hanya beberapa jam. Tidak ada alasan menundanya.

10. Apa yang Terjadi Jika Tidak Dikerjakan

Skenario ini disusun agar keputusan dapat diambil dengan gambaran yang jelas.

Jika tidak ada yang dikerjakan sama sekali

Waktu	Yang terjadi
Sekarang, pemakaian sedikit	Halaman terasa lambat ($\pm$ 10 detik), tapi jalan
Saat 20–30 orang bersamaan	Sebagian pengguna mulai melihat "Gagal memuat data"
Saat 50 orang bersamaan	Halaman Ringkasan tidak dapat digunakan. Refresh memperparah keadaan
Tahun anggaran 2027	Data dua kali lipat. Batas kegagalan turun ke sekitar 25 pengguna
Kapan saja	Satu file impor dengan format menyimpang → dashboard mati total sampai diperbaiki manual
Kapan saja	Angka realisasi bisa keliru tanpa peringatan, terbawa ke laporan resmi

Jika hanya Langkah 1 dan 4 yang dikerjakan (1,5 hari)

Waktu	Yang terjadi
50 orang bersamaan	✅ Berjalan normal
200 orang bersamaan	✅ Kemungkinan besar masih normal
Tahun anggaran 2027	✅ Tertangani (rekap dihitung sekali, bukan per permintaan)
File impor menyimpang	⚠️ Masih berisiko — tapi kini gagalnya saat pembaruan rekap, bukan saat pengguna membuka halaman. Pengguna masih melihat rekap terakhir yang berhasil , tidak melihat halaman mati

Waktu	Yang terjadi
Angka keliru diam-diam	⚠ Masih berisiko sampai Langkah 6 dikerjakan

Perhatikan baris keempat: bahkan tanpa memperbaiki masalah format data, Langkah 1 **mengubah kegagalan total menjadi kegagalan yang tidak terlihat pengguna**. Ini nilai tambah yang sering terlewat — rekap yang gagal diperbarui berarti data agak lama, bukan halaman mati.

Jika seluruh delapan langkah dikerjakan (7–9 hari)

Aspek	Kondisi
Jumlah pengguna	Ratusan, tanpa masalah
Pertumbuhan tahunan	Tertangani
Risiko kegagalan impor	Hilang (ada pemeriksaan format)
Risiko angka keliru	Hilang
Risiko pengembangan ke depan	Turun drastis — tidak ada lagi tambalan rapuh
Waktu buka halaman	Di bawah 1 detik

Lampiran A — Metode Pengukuran

Seluruh angka dalam laporan ini berasal dari **pengukuran langsung** ke sistem yang sedang berjalan pada 5–6 Agustus 2026, bukan dari perkiraan.

A.1. Jumlah baris data

Dihitung dengan meminta database melaporkan jumlah baris pada setiap tabel dan view yang diakses aplikasi.

Tabel / View	Jumlah baris
view_dashboard_gabungan_satker	7.734
risiko_pengadaan	7.741

Tabel / View	Jumlah baris
view_dashboard_pengadaan_langsung	6.023
view_dashboard_epurchasing_v6	1.521
view_dashboard_penunjukan_langsung	83
view_dashboard_tender	64
view_dashboard_swakelola_v1	43
master_data	88
data_afirmasi_pdn_perencanaan	44

A.2. Ukuran data yang dipindahkan

Diukur dengan mengambil 1.000 baris pertama dari setiap sumber dan mencatat ukuran sesungguhnya.

Sumber	Per 1.000 baris (mentah)	Per 1.000 baris (dipadatkan)
Paket gabungan	483 KB	40 KB
Risiko (dengan rincian komponen)	1.946 KB	53 KB

Angka "mentah" adalah yang harus diolah database dan browser. Angka "dipadatkan" adalah yang lewat jaringan. **Keduanya penting** — jaringan mungkin sanggup, tapi pengolahan di kedua ujung tetap membayar penuh.

Total untuk satu kali buka halaman Ringkasan:

- $8 \times 483 \text{ KB (paket)} + 8 \times 1.946 \text{ KB (risiko)} + \pm 1.000 \text{ KB (ITKP)} \approx \mathbf{19,7 \text{ MB mentah}}$

A.3. Waktu pengambilan

Diukur dari koneksi internet biasa. Waktu sesungguhnya bagi pengguna dapat berbeda tergantung jaringan, tapi **pola perbandingannya tetap berlaku**.

Permintaan	Waktu
Baris ke-1 s.d. 1.000	1,290 detik

Permintaan	Waktu
Baris ke-3.001 s.d. 4.000	1,316 detik
Baris ke-7.001 s.d. 7.734	1,477 detik

Cara membaca hasil ini: jika daftar isi gudang berfungsi, permintaan ketiga seharusnya jauh lebih cepat — sistem tinggal melompat ke bagian akhir. Kenyataannya ketiganya hampir sama, yang membuktikan **setiap permintaan membuka seluruh gudang dari nol** (lihat bagian 5.2).

A.4. Pemeriksaan kemampuan hitung di sisi database

Saya menguji apakah database sudah diizinkan melakukan penjumlahan langsung. Hasilnya: **belum diizinkan** — ada pengaturan yang masih tertutup.

Ini relevan untuk Langkah 3: mengaktifkan pengaturan tersebut membuka jalur pengerjaan yang lebih cepat sebagai alternatif.

A.5. Sumber pembacaan struktur

- `database.types.ts` — struktur lengkap seluruh tabel, view, dan fungsi
- Berkas SQL di folder `sql/` — definisi view dan catatan pengembang
- Kode aplikasi di folder `src/features/` — cara setiap halaman mengambil data

Lampiran B — Daftar Berkas Terkait

Berkas yang perlu diubah per langkah

Langkah	Berkas utama
1, 3	Berkas SQL baru di <code>sql/</code>
2	<code>src/features/ringkasan/lib/ringkasanData.ts</code> dan seluruh berkas <code>*View.tsx</code>
3	<code>src/features/ringkasan/lib/ringkasanData.ts</code>
4	<code>src/app/(app)/ringkasan/page.tsx</code> dan <code>RingkasanView.tsx</code>
5	<code>src/features/ringkasan/components/RisikoInsightPanel.tsx</code>

Langkah	Berkas utama
6	Seluruh berkas *View.tsx , ringkasanData.ts , src/lib/itkp/fetchA.ts
7	RingkasanView.tsx , ExportDataModal.tsx
8	Berkas SQL migrasi baru + seluruh berkas yang menyentuh nomor paket

Berkas yang wajib dibaca sebelum mulai

Berkas	Alasan
AGENTS.md	Menyatakan versi Next.js proyek ini berbeda dari umumnya — wajib dibaca sebelum Langkah 4
sql/add_index_realisasi_dashboard.sql	Berisi riwayat kegagalan sebelumnya dan rekomendasi tim
sql/add_index_ltrim_satker.sql	Menjelaskan mengapa daftar isi bisa mendadak tidak terpakai
sql/create_view_dashboard_pengadaan_langsung.sql	Berisi dua akar masalah terberat sekaligus

Lampiran C — Glosarium

Istilah teknis yang mungkin muncul dalam diskusi dengan tim pengembang.

Istilah teknis	Penjelasan sederhana	Bagian terkait
Query	Satu permintaan pengambilan data ke database	3.1
Index	Daftar isi database, agar pencarian tidak perlu membuka semua	5.1
Expression index	Daftar isi khusus yang hanya cocok jika pencariannya ditulis persis sama	5.2

Istilah teknis	Penjelasan sederhana	Bagian terkait
Full scan	Membuka seluruh isi rak satu per satu karena daftar isi tidak terpakai	5.2
View	Petunjuk kerja tersimpan: "ambil dari sini, gabungkan dengan itu" — dijalankan ulang setiap dipanggil	5.2
Materialized view	Rekap yang sudah dihitung dan disimpan permanen , tidak dijalankan ulang	Langkah 1
RPC / Function	Perintah tersimpan di database yang mengembalikan hasil jadi	Langkah 3
Cache	Simpanan sementara hasil perhitungan, agar tidak dihitung ulang	Langkah 4
Pagination / .range()	Mengambil data bertahap, 1.000 baris sekali angkut	3.1, 6.1
Timeout	Permintaan dibatalkan otomatis karena menunggu terlalu lama	4.2
Connection pool	Jumlah "pintu" database yang bisa dipakai bersamaan ($\pm$ 10–15)	4.2
JSONB	Cara menyimpan lampiran panjang di dalam satu kolom	5.3, Langkah 5
TOAST	Penyimpanan terpisah untuk lampiran besar — mengambilnya butuh kerja ekstra	Langkah 5
Bundle	Kumpulan program yang harus diunduh browser sebelum halaman jalan	3.1, 6.3
Dynamic import	Menunda pengunduhan program sampai benar-benar dibutuhkan	Langkah 7
Server Component	Halaman yang angkanya sudah disiapkan server sebelum dikirim ke browser	Langkah 4
RLS (Row Level Security)	Pembatasan akses data di tingkat database, bukan tampilan	6.4

Istilah teknis	Penjelasan sederhana	Bagian terkait
Foreign key	Pendaftaran hubungan resmi antar tabel	5.1
1NF	Aturan bahwa satu kolom hanya boleh berisi satu nilai (bukan "4521;4522")	5.2

Penutup

Halaman Ringkasan bukan halaman yang buruk. Fiturnya lengkap, perhitungannya teliti, dan tampilannya tertata. Masalahnya terletak pada **satu keputusan arsitektur di awal** — menghitung semuanya di browser pengguna — yang wajar untuk sistem dengan segelintir pengguna, tapi tidak bisa dipertahankan untuk 50 orang atau lebih.

Kabar baiknya, keputusan itu **tidak perlu dibongkar total**. Tampilan, perhitungan, dan logika bisnis yang sudah ada bisa dipertahankan hampir seluruhnya. Yang berubah hanyalah **di mana** perhitungan dilakukan: dari browser 50 pengguna, menjadi sekali di database.

Dan seperti terlihat di bagian 8.5, arah ini **sudah ditetapkan oleh tim sendiri** dalam catatan kode mereka. Laporan ini pada dasarnya hanya menegaskan bahwa waktunya sudah tiba, dan menyusun urutannya.

Belum ada kode yang diubah. Seluruh isi laporan ini adalah hasil pembacaan dan pengukuran.

Laporan disusun 6 Agustus 2026. Seluruh angka pengukuran mencerminkan kondisi sistem pada tanggal tersebut.